

DANIEL GOMIG

Arrays (Vetores)

Arrays são estruturas de dados onde a gente guarda vários valores do *mesmo tipo* um ao lado do outro na memória.

Um array tem tamanho fixo, ou seja, se você criar um array com 10 posições, ele vai ter sempre 10 posições.

Exemplo: uma lista com as notas de um aluno: 8, 7, 9, 10.

Vantagem: acesso rápido aos elementos (pela posição).

Desvantagem: não dá para mudar o tamanho.

Matrizes

Uma matriz é basicamente um *array de arrays*. Ela funciona como uma tabela, com linhas e colunas.

Exemplo: um tabuleiro de jogo da velha é uma matriz 3×3.

Vantagem: ótima para representar tabelas, imagens e mapas.

Desvantagem: também tem tamanho fixo.

Listas

Listas são parecidas com arrays, mas são *dinâmicas*, ou seja, podem crescer e diminuir.

Existem dois tipos principais:

Lista encadeada: cada elemento guarda o valor e o endereço do próximo.

Lista duplamente encadeada: cada elemento sabe quem é o próximo e quem é o anterior.

Vantagem: fácil inserir e remover elementos.

Desvantagem: acessar uma posição específica é mais demorado.

Pilhas (Stacks)

Uma pilha funciona como uma pilha de pratos: você só mexe no topo.

O termo técnico é LIFO — Last In, First Out (o último que entra é o primeiro que sai).

Operações principais:

push: colocar algo no topo

pop: retirar do topo

Filas (Queues)

Uma fila é como uma fila de supermercado: o primeiro que chega é o primeiro que sai, ou seja, FIFO — First In, First Out.

Exemplos: fila de impressão, processamento de tarefas por ordem de chegada

Operações:

enqueue: entrar na fila

dequeue: sair da fila

Mapas e Tabelas Hash (Hash Maps / Hash Tables)

Um mapa guarda pares do tipo chave → valor.

Por exemplo: "nome" → Daniel.

A tabela hash usa uma função chamada função hash, que transforma a chave em um número para ajudar a encontrar o valor rapidamente.

Vantagens: extremamente rápidas para buscar e armazenar coisas.

Desvantagens: podem sofrer colisões (quando duas chaves caem no mesmo lugar), mas existem técnicas para resolver isso.

Árvores

Uma árvore é uma estrutura em formato hierárquico, como uma árvore genealógica. Cada ponto da árvore é chamado de nó, e cada nó pode ter “filhos”. Um tipo muito famoso é a árvore binária, onde cada nó tem no máximo dois filhos.

Uso comum:

Organizar informações.

Buscas rápidas.

Sistemas de arquivos (pastas e subpastas)

Árvores balanceadas (como AVL e Red-Black) conseguem manter a velocidade rápida nas buscas.

Algoritmos de Ordenação

Algoritmos de ordenação servem para colocar os dados em ordem, como crescente ou alfabética.

Existem vários, alguns mais simples (e lentos) e outros bem eficientes.

Bubble Sort

Compara dois elementos de cada vez e troca se estiverem fora de ordem. É simples, mas bem lento.

Selection Sort

Procura o menor elemento e coloca no início.

Também é simples, mas lento para listas grandes.

Insertion Sort

Funciona como quando você organiza cartas na mão: vai colocando cada item no lugar certo. É rápido para listas pequenas.

Merge Sort

Divide a lista em partes menores, ordena cada parte e depois junta tudo.

É rápido e muito usado.

Quick Sort

Escolhe um elemento chamado *pivô* e organiza o resto ao redor dele.

É um dos mais rápidos na prática.

Algoritmos de Busca

Servem para encontrar um valor dentro de uma estrutura.

Busca Linear.

Passa item por item procurando o valor. Funciona em qualquer lista, mas pode ser lento.

Busca Binária

Só funciona em listas ordenadas.

Ela divide a lista ao meio e procura de forma muito rápida.

Exemplo:

Como procurar uma palavra no dicionário: você não começa na primeira página, você vai direto no meio.